

Theory of Programming Languages

7. Inductive Type

Kihong Heo



Motivation

- Let us extend simply typed λ -calculus with various inductively defined values
- E.g., boolean, natural number, list, tree, etc
- Example

```
type boolean = True | False
type nat = Zero | Succ of nat
type list = Nil | Cons of nat * list
type tree = Leaf of nat | Node of tree * tree
```

Inference Rules 추론 규칙

- Methods to **derive new facts from known facts** (i.e., derivable = provable)
- Notation of inference rule:

$$\frac{P_1 \ \cdots \ P_n \text{ premise(s)}}{P \text{ conclusion}} \qquad \frac{\text{human}(x) \implies \text{mortal}(x) \quad \text{human}(s)}{\text{mortal}(s)}$$

- Inference rule with zero premise is called an **axiom**
- Examples: inference rules of propositional logic

$$\frac{}{\text{true}} \quad \frac{A \quad B}{A \wedge B} \quad \frac{A \wedge B}{A} \quad \frac{A \wedge B}{B} \quad \frac{A}{A \vee B} \quad \frac{B}{A \vee B} \quad \frac{A \quad A \implies B}{B} \quad \dots$$

Inductive Definition of Set (1)

- Induction: a way to define a set (as well as a proof technique) using inference rules
 - E.g., natural number, list, binary tree, expression, etc
- Consider function ϕ derived from the set Φ of inference rules

$$\phi(Y) = \{x \mid \frac{X}{x} \in \Phi, X \subseteq Y\}$$

- Example:
 - $\phi(\{\}), \phi(\{0\}), \phi(\{3,5,7\})$

Inductive Definition of Set (2)

- Closed set (닫힌 집합): a set is **closed under an operation** if performing that operation on members of the set always produces a member of that set

- Inductively defined set: the least ϕ -closed set (i.e., closure, the least fixed point of ϕ)

$$S = \bigcap \{X \mid \phi(X) \subseteq X\}$$

- How to obtain such a set?

$$\phi^0 \cup \phi^1 \cup \phi^2 \cup \dots = \bigcup_{i \in \mathbb{N}} \phi^i$$

- Example

- $\mathbb{N}$
- list

Example

`type` boolean = True | False

$$\frac{}{\text{True}} \quad \frac{}{\text{False}}$$

`type` nat = Zero | Succ `of` nat

$$\frac{}{0} \quad \frac{n \in \text{nat}}{n + 1}$$

`type` list = Nil | Cons `of` nat * list

$$\frac{}{\text{nil}} \quad \frac{n \in \text{nat} \quad l \in \text{list}}{n :: l}$$

`type` tree = Leaf `of` nat | Node `of` tree * tree

$$\frac{n \in \text{nat}}{\text{leaf}(n)} \quad \frac{t_1 \in \text{tree} \quad t_2 \in \text{tree}}{\text{node}(t_1, t_2)}$$

System T

- Introduced by Gödel (1958)
- Simply typed λ -calculus + natural number
- Idea: introduction and elimination rules for the natural number type
 - Natural number: results of a finite composition of its introduction rules

Natural Number: Type

- Introduction

$$\frac{}{\Gamma \vdash 0 : \text{nat}} \quad \frac{\Gamma \vdash E : \text{nat}}{\Gamma \vdash s E : \text{nat}}$$

- Elimination (1st trial)

$$\frac{\Gamma \vdash E_1 : \text{nat} \quad \Gamma \vdash E_2 : \tau \quad \Gamma + x : \text{nat} \vdash E_3 : \tau}{\Gamma \vdash \text{case } E_1 \text{ of } 0 \Rightarrow E_2 \mid s x \Rightarrow E_3 : \tau}$$

Natural Number: Syntax & Dynamic

$$\begin{array}{lcl}
 E & \rightarrow & \dots \\
 & | & 0 \mid \mathbf{s} E \\
 & | & \text{case } E \text{ of } 0 \Rightarrow E \mid \mathbf{s} x \Rightarrow E \\
 K & \rightarrow & \dots \\
 & | & \mathbf{s} K \\
 & | & \text{case } K \text{ of } 0 \Rightarrow E \mid \mathbf{s} x \Rightarrow E \\
 v & \rightarrow & \dots \mid 0 \mid \mathbf{s} v
 \end{array}$$

$$\frac{E \rightarrow E'}{K[E] \rightarrow K[E']}$$

$$\frac{}{\text{case } 0 \text{ of } 0 \Rightarrow E_1 \mid \mathbf{s} x \Rightarrow E_2 \rightarrow E_1}$$

$$\frac{}{\text{case } \mathbf{s} v \text{ of } 0 \Rightarrow E_1 \mid \mathbf{s} x \Rightarrow E_2 \rightarrow E_2\{v/x\}}$$

Example

- Predecessor: $pred = \lambda x:\text{nat}. \text{case } x \text{ of } 0 \Rightarrow 0 \mid s \ y \Rightarrow y$
- Predecessor of 2: $pred(\text{succ}(\text{succ zero}))$
- Double?
 - `let double x = match x with Z -> Z | S y -> S (S (double y))`
 - Not possible! Why?

Primitive Recursion

- Generalized elimination construct for inductive types
 - E.g., natural number, list, tree, etc
- Primitive: recurse along the inductive structure of inductively defined types
 - Always terminate (general recursion may not terminate)
- Example: function double using primitive recursion in OCaml

```
let rec double x =  
  match x with  
  | Zero -> Zero  
  | Succ y -> Succ (Succ (double y))
```

Primitive Recursion for Natural Numbers (1)

- Generalized elimination construct for natural numbers
- Syntax
 - Recursor $\text{rec}(E, E_z, x . y . E_s)$: E -fold iteration starting from E_0
 - x : predecessor bound with scope E_s
 - y : the result of x -fold iteration (previous recursive call)

```
let rec f e =  
  match e with  
  | Zero -> (* do  $E_z$  *)  
  | Succ x ->  
    let y = f x in  
    (* do  $E_s$  *)
```

Primitive Recursion for Natural Numbers (2)

- Syntax & evaluation context

$$\begin{array}{c} E \rightarrow \dots \\ | \text{rec}(E, E_z, x.y.E_s) \end{array}$$

$$\begin{array}{c} K \rightarrow \dots \\ | \text{rec}(K, E_z, x.y.E_s) \end{array}$$

- Dynamic semantics

$$\frac{}{\text{rec}(0, E_z, x.y.E_s) \rightarrow E_z}$$

$$\frac{}{\text{rec}(s\ v, E_z, x.y.E_s) \rightarrow E_s\{v/x\}\{\text{rec}(v, E_z, x.y.E_s)/y\}}$$

- Static semantics

$$\frac{\Gamma \vdash E : \text{nat} \quad \Gamma \vdash E_z : \tau \quad \Gamma, x:\text{nat}, y:\tau \vdash E_s : \tau}{\Gamma \vdash \text{rec}(E, E_z, x.y.E_s) : \tau}$$

Exercise

- Double
- Plus
- Factorial

```
let rec double x =  
  match x with  
  | Zero -> Zero  
  | Succ y -> Succ (Succ (double y))
```

```
let rec plus x y =  
  match x with  
  | Zero -> y  
  | Succ x' -> Succ (plus x' y)
```

```
let rec factorial x =  
  match x with  
  | Zero -> Succ Zero  
  | Succ y -> times x (factorial y)
```

List

- Let us extend the language to support X lists
 - X : a fixed type such as nat, bool, etc (i.e., non-polymorphic list)
 - Polymorphic type will be discussed later
- Same principle: recursor on list

Primitive Recursion for List (1)

- Introduction

$$\frac{}{\Gamma \vdash \text{nil} : \tau \text{ list}} \quad \frac{\Gamma \vdash h : \tau \quad \Gamma \vdash t : \tau \text{ list}}{\Gamma \vdash h :: t : \tau \text{ list}}$$

- Elimination

$$\frac{\Gamma \vdash E : \tau \text{ list} \quad \Gamma \vdash E_n : \sigma \quad \Gamma, x:\tau, y:\tau \text{ list}, z:\sigma \vdash E_c : \sigma}{\Gamma \vdash \text{rec}(E, E_n, x.y.z.E_c) : \sigma}$$

Primitive Recursion for List (2)

- Syntax

$$\begin{array}{lcl} E & \rightarrow & \dots \\ & | & \text{nil} \mid E :: E \\ & | & \text{rec}(E, E, x.y.E) \\ K & \rightarrow & \dots \\ & | & K :: E \mid v :: K \\ & | & \text{rec}(K, E, x.y.E) \\ v & \rightarrow & \dots \mid \text{nil} \mid v :: v \end{array}$$

- Dynamic semantics

$$\frac{}{\text{rec}(\text{nil}, E_n, x.y.E_c) \rightarrow E_n}$$

$$\frac{}{\text{rec}(h :: t, E_n, x.y.E_c) \rightarrow E_c\{h/x\}\{t/y\}\{\text{rec}(t, E_n, x.y.z.E_c)/z\}}$$

Logic

$$\frac{\Gamma \vdash E : \text{nat} \quad \Gamma \vdash E_z : \tau \quad \Gamma, x:\text{nat}, y:\tau \vdash E_s : \tau}{\Gamma \vdash \text{rec}(E, E_z, x.y.E_s) : \tau}$$

- When is “simple type + primitive recursion” useful in proof?
- The current type system is not powerful enough for proving practical propositions
 - E.g., $\forall n, m \in \mathbb{N}. n < m \rightarrow n + 1 < m + 1$
- Need to extend type systems
 - How? Propositions depending on values, e.g., $\forall n \in \mathbb{N}. P(n, m) \rightarrow P(n + 1, m + 1)$
 - Equivalently, types depending on values (i.e., dependent type)

Recursor in Lean

```
inductive MyBool where  
  | true  
  | false
```

```
#check MyBool.rec
```

```
MyBool.rec.{u} {motive : MyBool → Sort u} (true : motive MyBool.true)  
(false : motive MyBool.false) (t : MyBool) : motive t
```

```
inductive MyNat where  
  | zero  
  | succ (n : MyNat)
```

```
#check MyNat.rec
```

```
MyNat.rec.{u} {motive : MyNat → Sort u} (zero : motive MyNat.zero)  
(succ : (n : MyNat) → motive n → motive n.succ) (t : MyNat) : motive t
```

Summary

- Inductive type: constructed by inductive definitions
- System T = Simply typed lambda calculus with inductive data types
- Primitive recursion: recurse over inductive structures of data types
- Strong normalization: all well-typed terms terminate
- Logic side: more powerful type systems needed for proof-by-induction